

Chapter 2 Cleaning Code for NHANES Diet Data

This chapter walks you through all of code used to clean the raw **NHANES dietary intake** datasets.

2.1 Processed and Organ Meat

To start, we need to disaggregate “proceessed meat” into “processed red meat” and “processed poultry” categories. We need this because the environmental impact factors don’t take “processed” into consideration, so we’ll need to eventually create new food groups for the environment model (“total red meat” and “total poultry”, which include both unprocessed and processed meat).

This is done using a SAS MACRO written by the USDA, located here:

- [TBD/Processed Meat Categories NHANES day 1_revised_121123.sas](#)

This code produces four SAS datasets:

- meat_day1.sas7bdat
- meat_day1_sum.sas7bdat
- meat_day2.sas7bdat
- meat_day2_sum.sas7bdat

We also have to disaggregate the organ meat food codes into “red meat” or “poultry” categories. I manually mapped various FNDDS codes that represented organ meats to either “red meat” or “poultry”. The mapping is here:

- [methods_manual/data_inputs/DIET/dietary_intake/DATA/raw_data/meat/organ_meats_bothdays_mapped_121323.csv](#)

Then, I merged this mapping with the processed meat data.

```

library(tidyverse)
library(haven)

rm(list = ls())

# read in meat data
meat_day1 <- read_sas("data_inputs/DIET/dietary_intake/DATA/clean_data/meat_day1.s
meat_day2 <- read_sas("data_inputs/DIET/dietary_intake/DATA/clean_data/meat_day2.s

# read in organ mapping
new_organ <- read_csv("data_inputs/DIET/dietary_intake/DATA/raw_data/meat/organ_me

# DAY 1 -----

meat_day1_1 <- meat_day1 %>% filter(!(is.na(SEQN))) %>% select(SEQN, DESCRIPTION,
                                                                DR1I_PF_ORGAN,
                                                                total_redmeat, tota

  arrange(SEQN, DR1ILINE)

# filter to organ intake > 0
meat_day1_1 %>% filter(DR1I_PF_ORGAN > 0)

# join with day1
meat_day1_2 <- left_join(meat_day1_1, new_organ, by = "DESCRIPTION")

meat_day1_3 <- meat_day1_2 %>%
  rowwise() %>%
  mutate(new = ifelse(is.na(new), "No change", new),
         total_redmeat_new = ifelse(new == "pf_redm", total_redmeat + DR1I_PF_ORGA
         total_poultry_new = ifelse(new == "pf_poultry", total_poultry + DR1I_PF_C
  select(SEQN, DR1ILINE, total_redmeat_new, total_poultry_new)

# export
write_rds(meat_day1_3, "data_inputs/DIET/dietary_intake/DATA/clean_data/meat_day1.

```

```

# DAY 2 -----

meat_day2_1 <- meat_day2 %>% filter(!is.na(SEQN)) %>% select(SEQN, DESCRIPTION2,
                                                             DR2I_PF_ORGAN, total_redmeat_day2, total_poultry_day2) %>%
  arrange(SEQN, DR2ILINE)

meat_day2_1 %>% filter(DR2I_PF_ORGAN > 0)

# join with day2
meat_day2_2 <- left_join(meat_day2_1, new_organ, by = c("DESCRIPTION2" = "DESCRIPTION2", "DR2I_PF_ORGAN" = "DR2I_PF_ORGAN"))

meat_day2_3 <- meat_day2_2 %>%
  rowwise() %>%
  mutate(new = ifelse(is.na(new), "No change", new),
         total_redmeat_new = ifelse(new == "pf_redm", total_redmeat_day2 + DR2I_PF_ORGAN, total_redmeat_day2),
         total_poultry_new = ifelse(new == "pf_poultry", total_poultry_day2 + DR2I_PF_ORGAN, total_poultry_day2)) %>%
  select(SEQN, DR2ILINE, total_redmeat_new, total_poultry_new)

# export
write_rds(meat_day2_3, "data_inputs/DIET/dietary_intake/DATA/clean_data/meat_day2.rds")

```

2.2 Clean Raw NHANES Data

Now that I have dealt with processed and organ meat, I can start to clean the raw NHANES data. We use data from the 2015-2016 and 2017-2018 cycles.

Step 1: Set up workspace

```
rm(list=ls())  
  
# Load packages  
library(foreign)  
library(survey)  
library(tidyverse)  
library(psych)  
library(haven)  
library(readxl)
```

Step 2: Clean individual-level food intake datasets

```
# 2015-2016 diet data (i)

# foods day 1
foods_i1_nutrients <- read_sas("data_inputs/DIET/dietary_intake/DATA/raw_data/dr1i

foods_i1_whole <- read_sas("data_inputs/DIET/dietary_intake/DATA/raw_data/fped_dr1

# join
foods_i1 <- left_join(foods_i1_whole, foods_i1_nutrients) %>% mutate(nhanes_cycle

# foods day 2
foods_i2_nutrients <- read_sas("data_inputs/DIET/dietary_intake/DATA/raw_data/dr2i

foods_i2_whole <- read_sas("data_inputs/DIET/dietary_intake/DATA/raw_data/fped_dr2

# join
foods_i2 <- left_join(foods_i2_whole, foods_i2_nutrients) %>% mutate(nhanes_cycle

# 2017-2018 diet data (j)

# foods day 1
foods_j1_nutrients <- read_sas("data_inputs/DIET/dietary_intake/DATA/raw_data/dr1i

foods_j1_whole <- read_sas("data_inputs/DIET/dietary_intake/DATA/raw_data/fped_dr1

#join
foods_j1 <- left_join(foods_j1_whole, foods_j1_nutrients) %>% mutate(nhanes_cycle

# foods day 2
foods_j2_nutrients <- read_sas("data_inputs/DIET/dietary_intake/DATA/raw_data/dr2i

foods_j2_whole <- read_sas("data_inputs/DIET/dietary_intake/DATA/raw_data/fped_dr2
```

```

# join

foods_j2 <- left_join(foods_j2_whole, foods_j2_nutrients) %>% mutate(nhanes_cycle

# create day 1 and 2 datasets

foods_day1 <- rbind(foods_i1, foods_j1) %>%

  mutate(foodsource = ifelse(DR1FS == 1, "Grocery", "Other"), # create food source
         foodsource = replace_na(foodsource, "Other"), # replace NAs with "other"
         dayrec = 1) # day1

foods_day2 <- rbind(foods_i2, foods_j2) %>%

  mutate(foodsource = ifelse(DR2FS == 1, "Grocery", "Other"), # create food source
         foodsource = replace_na(foodsource, "Other"), # replace NAs with "other"
         dayrec = 2)

# read in meat data

meat_day1 <- read_rds("data_inputs/DIET/dietary_intake/DATA/clean_data/meat_day1.r
meat_day2 <- read_rds("data_inputs/DIET/dietary_intake/DATA/clean_data/meat_day2.r

foods_day1_ <- left_join(foods_day1, meat_day1, by = c("SEQN", "DR1ILINE"))
foods_day2_ <- left_join(foods_day2, meat_day2, by = c("SEQN", "DR2ILINE"))

# check

foods_day1_ %>% select(SEQN, DR1ILINE, DR1I_PF_MPS_TOTAL, DR1I_PF_SEAFD_HI, DR1I_F

foods_day1_ %>% filter(DR1I_PF_ORGAN > 0) %>% select(SEQN, DR1ILINE, DR1I_PF_MPS_I

# Calculate amount of intake for each dietary factor
# for day 1 and day 2
# by SEQN and foodsource

# first need to create diet variables that are combinations of 2+ vars

# day 1

foods_day1_1 <- foods_day1_ %>%

rowwise() %>%

```

```

rename(sat_fat = DR1ISFAT,
       p_fat = DR1IPFAT,
       sodium = DR1ISODI,
       gr_refined = DR1I_G_REFINED,
       gr_whole = DR1I_G_WHOLE,
       added_sugar = DR1I_ADD_SUGARS,
       fruit_tot = DR1I_F_TOTAL,
       fruit_juice = DR1I_F_JUICE,
       fiber = DR1IFIBE,
       dairy_tot = DR1I_D_TOTAL,
       veg_dg = DR1I_V_DRKGR,
       veg_oth = DR1I_V_OTHER,
       veg_ro = DR1I_V_REDOR_TOTAL,
       veg_sta = DR1I_V_STARCHY_TOTAL,
       veg_leg = DR1I_V_LEGUMES,
       oil = DR1I_OILS,
       pf_egg = DR1I_PF_EGGS,
       pf_ns = DR1I_PF_NUTSDS,
       pf_soy = DR1I_PF_SOY,
       pf_poultry = DR1I_PF_POULT,
       pf_redm = DR1I_PF_MEAT,
       pf_redm_tot = total_redmeat_new,
       pf_poultry_tot = total_poultry_new,
       pf_leg = DR1I_PF_LEGUMES,
       kcal = DR1IKCAL) %>%

mutate(sea_omega3_fa = sum(DR1IP226, DR1IP205),
       veg_exc_sta = sum(veg_dg, veg_ro, veg_oth),
       fruit_exc_juice = sum(DR1I_F_CITMLB, DR1I_F_OTHER),
       pf_pm = sum(DR1I_PF_CUREDMEAT, DR1I_PF_ORGAN),
       pf_seafood = sum(DR1I_PF_SEAFD_HI, DR1I_PF_SEAFD_LOW),
       leg_tot = sum(pf_leg, pf_soy), # doesn't include soy milk?
       pf_animal = sum(DR1I_PF_MPS_TOTAL, pf_egg),
       pf_plant = sum(pf_leg, pf_ns, pf_soy)) %>%

```

```

ungroup()

# create soy milk category
foods_day1_2 <-
  foods_day1_1 %>%
  mutate(dairy_soy = ifelse(str_detect(DESCRIPTION, "Soy milk") & dairy_tot > 0,
                              dairy_tot,
                              0),
         dairy_cow = ifelse(str_detect(DESCRIPTION, "Soy milk", negate = TRUE) & c
                              dairy_tot,
                              0))

# check
foods_day1_2 %>%
  filter(dairy_tot > 0) %>%
  select(SEQN, DESCRIPTION, dairy_tot, dairy_soy, dairy_cow) #good

# day 2

foods_day2_1 <- foods_day2_ %>%

rowwise() %>%

rename(sat_fat = DR2ISFAT,
       p_fat = DR2IPFAT,
       sodium = DR2ISODI,
       gr_refined = DR2I_G_REFINED,
       gr_whole = DR2I_G_WHOLE,
       added_sugar = DR2I_ADD_SUGARS,
       fruit_tot = DR2I_F_TOTAL,
       fruit_juice = DR2I_F_JUICE,
       fiber = DR2IFIBE,
       dairy_tot = DR2I_D_TOTAL,
       veg_dg = DR2I_V_DRKGR,
       veg_oth = DR2I_V_OTHER,
       veg_ro = DR2I_V_REDOR_TOTAL,
       veg_sta = DR2I_V_STARCHY_TOTAL,

```



```

veg_leg = DR2I_V_LEGUMES,
oil = DR2I_OILS,
pf_egg = DR2I_PF_EGGS,
pf_ns = DR2I_PF_NUTSDS,
pf_soy = DR2I_PF_SOY,
pf_poultry = DR2I_PF_POULT,
pf_redm = DR2I_PF_MEAT,
pf_redm_tot = total_redmeat_new,
pf_poultry_tot = total_poultry_new,
pf_leg = DR2I_PF_LEGUMES,
kcal = DR2IKCAL) %>%

mutate(sea_omega3_fa = sum(DR2IP226, DR2IP205),
       veg_exc_sta = sum(veg_dg, veg_ro, veg_oth),
       fruit_exc_juice = sum(DR2I_F_CITMLB, DR2I_F_OTHER),
       pf_pm = sum(DR2I_PF_CUREDMEAT, DR2I_PF_ORGAN),
       pf_seafood = sum(DR2I_PF_SEAFD_HI, DR2I_PF_SEAFD_LOW),
       leg_tot = sum(pf_leg, pf_soy),
       pf_animal = sum(DR2I_PF_MPS_TOTAL, pf_egg),
       pf_plant = sum(pf_leg, pf_ns, pf_soy)) %>%

ungroup()

# create soy milk category
foods_day2_2 <-
  foods_day2_1 %>%
  mutate(dairy_soy = ifelse(str_detect(DESCRIPTION, "Soy milk") & dairy_tot > 0,
                             dairy_tot,
                             0),
         dairy_cow = ifelse(str_detect(DESCRIPTION, "Soy milk", negate = TRUE) & c
                             dairy_tot,
                             0))

# check
foods_day2_2 %>%
  filter(dairy_tot > 0) %>%
  select(SEQN, DESCRIPTION, dairy_tot, dairy_soy, dairy_cow) #good

```

```
# export foods day 1 and day 2 for later use  
write_rds(foods_day1_2, "data_inputs/DIET/dietary_intake/DATA/clean_data/foods_day  
write_rds(foods_day2_2, "data_inputs/DIET/dietary_intake/DATA/clean_data/foods_day
```

Step 3: Clean the food and nutrient intake (summary) datasets

```
rm(list=setdiff(ls(), c("foods_day1_2", "foods_day2_2")))

# QUICKLY HANDLE MEAT
meat_sum_day1 <-
  foods_day1_2 %>%
  group_by(SEQN) %>%
  summarise(pf_redm_tot_1 = sum(pf_redm_tot),
            pf_poultry_tot_1 = sum(pf_poultry_tot))

meat_sum_day2 <-
  foods_day2_2 %>%
  group_by(SEQN) %>%
  summarise(pf_redm_tot_2 = sum(pf_redm_tot),
            pf_poultry_tot_2 = sum(pf_poultry_tot))

# 2015-2016 diet data (i)

# demographic data
demo_i <- read_sas("data_inputs/DIET/dietary_intake/DATA/raw_data/demo_i.sas7bdat"
  select(SEQN, RIAGENDR, RIDRETH1, DMDEDUC2, INDFMPIR, RIDAGEYR)

# fped day 1
fped_i1 <- read_sas("data_inputs/DIET/dietary_intake/DATA/raw_data/fped_dr1tot_151

# fped day 2
fped_i2 <- read_sas("data_inputs/DIET/dietary_intake/DATA/raw_data/fped_dr2tot_151

# join the two datasets
fped_i <- left_join(fped_i1, fped_i2)

# nutrients day 1
nutrient_i1 <- read_sas("data_inputs/DIET/dietary_intake/DATA/raw_data/dr1tot_i.sa
```

```
# nutrients day 2
nutrient_i2 <- read_sas("data_inputs/DIET/dietary_intake/DATA/raw_data/dr2tot_i.sas")

# join the two datasets
nutrient_i <- full_join(nutrient_i1, nutrient_i2)

# Combine all datasets
nhanes1516 <-
  fped_i %>%
  left_join(nutrient_i) %>%
  left_join(demo_i)

# 2017-2018 diet data (j)

# demographic data
demo_j <- read_sas("data_inputs/DIET/dietary_intake/DATA/raw_data/demo_j.sas7bdat")
  select(SEQN, RIAGENDR, RIDRETH1, DMDDEDUC2, INDFMPIR, RIDAGEYR)

# fped day 1
fped_j1 <- read_sas("data_inputs/DIET/dietary_intake/DATA/raw_data/fped_dr1tot_171")

# fped day 2
fped_j2 <- read_sas("data_inputs/DIET/dietary_intake/DATA/raw_data/fped_dr2tot_171")

# join the two datasets
fped_j <- left_join(fped_j1, fped_j2)

# nutrients day 1
nutrient_j1 <- read_sas("data_inputs/DIET/dietary_intake/DATA/raw_data/dr1tot_j.sas")

# nutrients day 2
nutrient_j2 <- read_sas("data_inputs/DIET/dietary_intake/DATA/raw_data/dr2tot_j.sas")

# join the two datasets
nutrient_j <- full_join(nutrient_j1, nutrient_j2)
```

```

# Combine all datasets
nhanes1718 <- fped_j %>%
  left_join(nutrient_j) %>%
  left_join(demo_j)

# combine the two nhanes datasets

# first, change 2 variable names that don't match
nhanes1718 <- nhanes1718 %>% rename(DR1TWS = DR1TWSZ,
                                   DR2TWS = DR2TWSZ)

nhanes_comb <- rbind(nhanes1516, nhanes1718)

# combine with meat
nhanes_comb1 <- left_join(nhanes_comb, meat_sum_day1, by = "SEQN") %>% left_join(n

# create dairy variables
dairy_day1 <-
  foods_day1_2 %>%
  group_by(SEQN) %>%
  summarise(dairy_cow_1 = sum(dairy_cow),
            dairy_soy_1 = sum(dairy_soy))

dairy_day2 <-
  foods_day2_2 %>%
  group_by(SEQN) %>%
  summarise(dairy_cow_2 = sum(dairy_cow),
            dairy_soy_2 = sum(dairy_soy))

# merge with nhanes
dairy_bothdays <- full_join(dairy_day1, dairy_day2)

nhanes_comb2 <- nhanes_comb1 %>% left_join(dairy_bothdays, by = "SEQN")

```

Step 4: Construct dietary factors

```
nhanes_comb3 <-  
  nhanes_comb2 %>%  
  rename(kcal_1 = DR1TKCAL,  
         kcal_2 = DR2TKCAL,  
  
         sat_fat_1 = DR1TSFAT,  
         sat_fat_2 = DR2TSFAT,  
  
         sodium_1 = DR1TSODI,  
         sodium_2 = DR2TSODI,  
  
         gr_refined_1 = DR1T_G_REFINED,  
         gr_refined_2 = DR2T_G_REFINED,  
  
         gr_whole_1 = DR1T_G_WHOLE,  
         gr_whole_2 = DR2T_G_WHOLE,  
  
         added_sugar_1 = DR1T_ADD_SUGARS,  
         added_sugar_2 = DR2T_ADD_SUGARS,  
  
         fruit_tot_1 = DR1T_F_TOTAL,  
         fruit_tot_2 = DR2T_F_TOTAL,  
  
         fruit_juice_1 = DR1T_F_JUICE,  
         fruit_juice_2 = DR2T_F_JUICE,  
  
         fiber_1 = DR1TFIBE,  
         fiber_2 = DR2TFIBE,  
  
         dairy_tot_1 = DR1T_D_TOTAL,  
         dairy_tot_2 = DR2T_D_TOTAL,  
  
         veg_dg_1 = DR1T_V_DRKGR,
```

```
veg_dg_2 = DR2T_V_DRKGR,

veg_oth_1 = DR1T_V_OTHER,
veg_oth_2 = DR2T_V_OTHER,

veg_ro_1 = DR1T_V_REDOR_TOTAL,
veg_ro_2 = DR2T_V_REDOR_TOTAL,

veg_sta_1 = DR1T_V_STARCHY_TOTAL,
veg_sta_2 = DR2T_V_STARCHY_TOTAL,

# Beans, peas, and lentils (legumes) computed as vegetables (cup eq.)
veg_leg_1 = DR1T_V_LEGUMES,
veg_leg_2 = DR2T_V_LEGUMES,

oil_1 = DR1T_OILS,
oil_2 = DR2T_OILS,

pf_egg_1 = DR1T_PF_EGGS,
pf_egg_2 = DR2T_PF_EGGS,

pf_ns_1 = DR1T_PF_NUTSDS,
pf_ns_2 = DR2T_PF_NUTSDS,

# soy only
# Soy products, excluding calcium fortified soy milk (soymilk)
# and raw soybeans products (oz. eq.)
pf_soy_1 = DR1T_PF_SOY,
pf_soy_2 = DR2T_PF_SOY,

pf_poultry_1 = DR1T_PF_POULT,
pf_poultry_2 = DR2T_PF_POULT,

pf_redm_1 = DR1T_PF_MEAT,
pf_redm_2 = DR2T_PF_MEAT,
```

```

pf_pm_1 = DR1T_PF_CUREDMEAT,
pf_pm_2 = DR2T_PF_CUREDMEAT,

pf_organ_1 = DR1T_PF_ORGAN,
pf_organ_2 = DR2T_PF_ORGAN,

# Beans and Peas (legumes) computed as protein foods (oz. eq.)
pf_leg_1 = DR1T_PF_LEGUMES,
pf_leg_2 = DR2T_PF_LEGUMES) %>%

rowwise() %>%

mutate(sea_omega3_fa_1 = sum(DR1TP226, DR1TP205),
       sea_omega3_fa_2 = sum(DR2TP226, DR2TP205),

       veg_exc_sta_1 = sum(veg_dg_1, veg_ro_1, veg_oth_1),
       veg_exc_sta_2 = sum(veg_dg_2, veg_ro_2, veg_oth_2),

       fruit_exc_juice_1 = sum(DR1T_F_CITMLB, DR1T_F_OTHER),
       fruit_exc_juice_2 = sum(DR2T_F_CITMLB, DR2T_F_OTHER),

       # pf_redm_tot_1 = sum(),
       # pf_redm_tot_2 = sum(),
       #
       # pf_poultry_tot_1 = sum(),
       # pf_poultry_tot_2 = sum(),

       puфа_energy_1 = ((DR1TPFAT * 9) / kcal_1) * 100,
       puфа_energy_2 = ((DR2TPFAT * 9) / kcal_2) * 100,

       sfat_energy_1 = ((sat_fat_1 * 9) / kcal_1) * 100,
       sfat_energy_2 = ((sat_fat_2 * 9) / kcal_2) * 100,

       pf_seafood_1 = sum(DR1T_PF_SEAFD_HI, DR1T_PF_SEAFD_LOW),
       pf_seafood_2 = sum(DR2T_PF_SEAFD_HI, DR2T_PF_SEAFD_LOW),

```



```

# includes legumes and soy foods
leg_tot_1 = sum(pf_leg_1, pf_soy_1),
leg_tot_2 = sum(pf_leg_2, pf_soy_2),

pf_animal_1 = sum(DR1T_PF_MPS_TOTAL, pf_egg_1),
pf_animal_2 = sum(DR2T_PF_MPS_TOTAL, pf_egg_2),

pf_plant_1 = sum(pf_leg_1, pf_ns_1, pf_soy_1),
pf_plant_2 = sum(pf_leg_2, pf_ns_2, pf_soy_2)
)

# select the variables we need
nhanes_comb4 <- nhanes_comb3 %>% select(SEQN, RIAGENDR, RIDRETH1, DMDEDUC2,
                                         INDFMPIR, RIDAGEYR, SDMVPSU, SDMVSTRA,
                                         WTDRD1, WTDR2D, DR1DRSTZ, DR2DRSTZ,
                                         DRDINT, ends_with("_1"), ends_with("_2"))

ungroup() # stop using rowwise

```

Step 5: Create sociodemographic subgroups

```
nhanes_comb5 <- nhanes_comb4 %>% mutate(  
  
  female = ifelse(RIAGENDR == 2, 1, 0),  
  
  sex = ifelse(female == 1, 1, 2),  
  
  race = recode(RIDRETH1,  
    `3` = 1,  
    `4` = 2,  
    `1` = 3,  
    `2` = 3,  
    `5` = 4),  
  
  age = case_when(RIDAGEYR >= 20 & RIDAGEYR < 35 ~ 1,  
    RIDAGEYR >= 35 & RIDAGEYR < 45 ~ 2,  
    RIDAGEYR >= 45 & RIDAGEYR < 55 ~ 3,  
    RIDAGEYR >= 55 & RIDAGEYR < 65 ~ 4,  
    RIDAGEYR >= 65 & RIDAGEYR < 75 ~ 5,  
    RIDAGEYR >= 75 ~ 6),  
  
  # create new weight variable  
  wtnew = WTD1/2)
```

Step 6: Create averages of dietary factors for Day 1 and Day 2

```
nhanes_comb6 <- nhanes_comb5 %>%
  rowwise() %>%
  mutate(kcal = mean(c(kcal_1, kcal_2), na.rm = TRUE),
         sat_fat = mean(c(sat_fat_1, sat_fat_2), na.rm = TRUE),
         sodium = mean(c(sodium_1, sodium_2), na.rm = TRUE),
         gr_refined = mean(c(gr_refined_1, gr_refined_2), na.rm = TRUE),
         gr_whole = mean(c(gr_whole_1, gr_whole_2), na.rm = TRUE),
         added_sugar = mean(c(added_sugar_1, added_sugar_2), na.rm = TRUE),
         fruit_tot = mean(c(fruit_tot_1, fruit_tot_2), na.rm = TRUE),
         fruit_exc_juice = mean(c(fruit_exc_juice_1, fruit_exc_juice_2), na.rm = TRUE),
         fruit_juice = mean(c(fruit_juice_1, fruit_juice_2), na.rm = TRUE),
         fiber = mean(c(fiber_1, fiber_2), na.rm = TRUE),
         dairy_tot = mean(c(dairy_tot_1, dairy_tot_2), na.rm = TRUE),
         dairy_cow = mean(c(dairy_cow_1, dairy_cow_2), na.rm = TRUE),
         dairy_soy = mean(c(dairy_soy_1, dairy_soy_2), na.rm = TRUE),
         veg_dg = mean(c(veg_dg_1, veg_dg_2), na.rm = TRUE),
         veg_oth = mean(c(veg_oth_1, veg_oth_2), na.rm = TRUE),
         veg_ro = mean(c(veg_ro_1, veg_ro_2), na.rm = TRUE),
         veg_sta = mean(c(veg_sta_1, veg_sta_2), na.rm = TRUE),
         veg_leg = mean(c(veg_leg_1, veg_leg_2), na.rm = TRUE),
         veg_exc_sta = mean(c(veg_exc_sta_1, veg_exc_sta_2), na.rm = TRUE),
         oil = mean(c(oil_1, oil_2), na.rm = TRUE),
         pf_egg = mean(c(pf_egg_1, pf_egg_2), na.rm = TRUE),
         pf_ns = mean(c(pf_ns_1, pf_ns_2), na.rm = TRUE),
         pf_soy = mean(c(pf_soy_1, pf_soy_2), na.rm = TRUE),
         pf_poultry = mean(c(pf_poultry_1, pf_poultry_2), na.rm = TRUE),
         pf_poultry_tot = mean(c(pf_poultry_tot_1, pf_poultry_tot_2), na.rm = TRUE),
         pf_pm = mean(c(pf_pm_1, pf_pm_2), na.rm = TRUE),
         pf_redm = mean(c(pf_redm_1, pf_redm_2), na.rm = TRUE),
         pf_redm_tot = mean(c(pf_redm_tot_1, pf_redm_tot_2), na.rm = TRUE),
         pf_organ = mean(c(pf_organ_1, pf_organ_2), na.rm = TRUE),
         pf_leg = mean(c(pf_leg_1, pf_leg_2), na.rm = TRUE),
         sea_omega3_fa = mean(c(sea_omega3_fa_1, sea_omega3_fa_2), na.rm = TRUE),
```

```

pufa_energy = mean(c(pufa_energy_1, pufa_energy_2), na.rm = TRUE),
sfat_energy = mean(c(sfat_energy_1, sfat_energy_2), na.rm = TRUE),
pf_seafood = mean(c(pf_seafood_1, pf_seafood_2), na.rm = TRUE),
leg_tot = mean(c(leg_tot_1, leg_tot_2), na.rm = TRUE),
pf_animal = mean(c(pf_animal_1, pf_animal_2), na.rm = TRUE),
pf_plant = mean(c(pf_plant_1, pf_plant_2), na.rm = TRUE))

# get rid of NaN
nhanes_comb6[nhanes_comb6 == "NaN"] <- NA

# ungroup
nhanes_comb7 <- nhanes_comb6 %>% ungroup()

# Merge with subgroup file
subgroups <- read_csv("data_inputs/OTHER/labels/DATA/population_subgroups_48_06092

nhanes_comb8 <-
  nhanes_comb7 %>%
  left_join(subgroups, by = c("age" = "Age",
                             "sex" = "Sex",
                             "race" = "Race"))

# create final dataset
nhanes_final <- nhanes_comb8

# Look at meat variables
nhanes_final %>%
  select(SEQN, starts_with(c("pf_redm", "pf_poultry", "pf_pm", "pf_organ")) %>%
  mutate(sumtot = (pf_redm + pf_poultry + pf_pm + pf_organ == pf_redm_tot + pf_pou
    sum1 = (pf_redm_1 + pf_poultry_1 + pf_pm_1 + pf_organ_1 == pf_redm_tot_1
    sum2 = (pf_redm_2 + pf_poultry_2 + pf_pm_2 + pf_organ_2 == pf_redm_tot_2

nhanes_final %>%
  select(SEQN, starts_with(c("pf_redm", "pf_poultry", "pf_pm", "pf_organ")) %>%
  mutate(sum1 = (pf_redm + pf_poultry + pf_pm + pf_organ == pf_redm_tot + pf_poult
  filter(sum1 == "FALSE")

```

```
# Look at day 1
```

```
nhanes_final %>%
  rowwise() %>%
  select(SEQN, pf_redm_1, pf_poultry_1, pf_pm_1, pf_organ_1, pf_redm_tot_1, pf_pou
  mutate(sum1 = round(sum(pf_redm_1, pf_poultry_1, pf_pm_1, pf_organ_1), digits =
    sum2 = round(sum(pf_redm_tot_1, pf_poultry_tot_1), digits = 3),
    my_test = (sum1 == sum2)) %>%
  filter(my_test == "FALSE") %>%
  mutate(my_subtract = sum2 - sum1) %>%
  filter(my_subtract < -0.05 | my_subtract > 0.05) # not bad
```

```
# Look at day2
```

```
nhanes_final %>%
  rowwise() %>%
  select(SEQN, pf_redm_2, pf_poultry_2, pf_pm_2, pf_organ_2, pf_redm_tot_2, pf_pou
  mutate(sum1 = round(sum(pf_redm_2, pf_poultry_2, pf_pm_2, pf_organ_2), digits =
    sum2 = round(sum(pf_redm_tot_2, pf_poultry_tot_2), digits = 3),
    my_test = (sum1 == sum2)) %>%
  filter(my_test == "FALSE") %>%
  mutate(my_subtract = sum2 - sum1) %>%
  filter(my_subtract < -0.05 | my_subtract > 0.05) # not bad
```

Step 7: Examine cleaned NHANES data

```

# check missing
summary(nhanes_final)

# check for outliers

# kcal < 500
nhanes_final %>% filter(kcal_1 < 500) %>% nrow() #233
nhanes_final %>% filter(kcal_2 < 500) %>% nrow() #301

# kcal > 3500
nhanes_final %>% filter(kcal_1 > 3500) %>% nrow() #1028
nhanes_final %>% filter(kcal_2 > 3500) %>% nrow() #588

# do not remove because I will filter out
# dietary recalls that are not valid later

# diet recall status
table(nhanes_final$DR1DRSTZ, useNA = "always")
table(nhanes_final$DR2DRSTZ, useNA = "always")

# first, create inAnalysis variable
nhanes_final1 <- nhanes_final %>%
  rowwise() %>%
  mutate(
    # Define sub-population of interest:
    # Adults aged 20+ with 1 or 2 days of reliable dietary recalls

    reliable_yes = ifelse((DRDINT == 1 & DR1DRSTZ == 1) | (DRDINT == 2 & DR1DRSTZ
    == 1), 1, 0),

    inAnalysis = (!is.na(subgroup)) & reliable_yes == 1, # if subgroup ISN'T mis

    # Change NAs to 0s, otherwise svydesign function below won't run
    wtnew = ifelse(is.na(wtnew), 0, wtnew),
  )

```

```
SDMVPSU = ifelse(is.na(SDMVPSU), 0, SDMVPSU),  
SDMVSTRA = ifelse(is.na(SDMVSTRA), 0, SDMVSTRA)  
)  
  
# check new survey weight  
nhanes_final1 %>% select(SEQN, wtnew, SDMVPSU, SDMVSTRA, inAnalysis) # Looks good  
  
# export  
write_rds(nhanes_final1,  
          "data_inputs/DIET/dietary_intake/DATA/clean_data/nhanes1518_clean.rds")
```

Finally, the cleaned dataset is exported to use later.